

哈尔滨工业大学

编译系统实验报告（二）

姓名	李凡丁
学号	7203610625
学院	计算机科学与技术
教师	朱庆福

2023 年 4 月

一、实验目的

1. 巩固对语义分析的基本功能和原理的认识。
2. 能够基于语法指导翻译的知识进行语义分析。
3. 理解并处理语义分析中的异常和错误。

二、实验环境

1. Ubuntu 20.04
2. GCC 9.4.0
3. Flex 2.6.4
4. Bison 3.5.1

三、实验过程

1. 实验内容

本次实验完成了对于 C—语言的语义分析，对于语义错误的检测。

2. 数据结构

本次实验是承接实验 1 进行的，所以采用的数据结构和实验 1 一样，同为语法分析树。在本次实验中新增的数据结构有两个：变量类型 Type 以及字符表中的一个域 FieldList。这两个数据结构按照参考书规定的方法进行编写即可，在此不过多赘述。

3. 语义分析过程

本次实验我们默认 C—文法中词法和语法是不发生错误的。这样我们需要先利用实验 1 的相关内容进行词法分析和语法分析，以便验证词法和语法的正确性，同时我们也能够在这个过程中构造语法树，从而开始我们的语义分析过程。

在实验 1 中，当我们正确构造语法树之后，我们会对于语法树进行遍历，输出每一部分的成分。在本实验中，我们同样需要进行语法树的遍历来实现语义分析。我们在遍历语法书的过程中，依照不同类型的错误原因展开分析，将这种错误对应到语法树上，这样当我们遍历语法树的时候碰到了相同的情况，这就可以作为错误评判的依据。下面展示一些错误的情况的判定方法：

(1) 错误类型 13：对非结构体类型变量使用“.”操作符。

```
if(strcmp(fir_bro->Token,"DOT")==0){
    //Exp → Exp DOT ID
    Type return_type=Exp(child);
    if(return_type==NULL) return NULL;
    if(return_type->type==STRUCT);
    else {
        printf("Error type 13 at Line %d: Illegal use of \".\".\n",now->line);
        return NULL;
    }
}
```

如上图代码所示，我们首先会检索到语法单元“.”，然后我们需要在语法分析树上进行回溯，判断其前一个语法单元是不是被定义为了结构体，如果是那么就是合法使用，反之就是非法调用。

由于我们使用的二叉树实现多叉树，并且“.”与其之前的语法单元在语法分析树中应当是兄弟节点关系，所以在我们实现的数据结构中，“.”之前的语法单元是其左子节点。我们只需要判断其 type 是不是 struct 就可以了。

(2) 错误类型 1：变量在使用时未经定义。

```
else if(strcmp(child->Token,"ID")==0){
    //Exp → ID
    //printf("aExp → ID\n");
    Vari find_id=find_vari_table(child->Id_Type);
    if(find_id==NULL|| find_id->is_def_struct == DEF_IN_STRUCT){
        printf("Error type 1 at Line %d: Undefined variable \"%s\".\n",now->line,child->Id_Type);
        return NULL;
    }
}
```

strcmp 函数返回值为 0 的时候证明二者是相等的，这里说明该语法单元是一个 ID 类型的语法单元。这样我们就在其类型的符号表中查找是不是存在这个变量的名称即可。当然也有特殊情况：当这个变量的定义在结构体中的时候，符号表中是存在这个变量的，但是这种是并不合法的，所以这种情况我们也要单独进行判断。

如果我们没有找到这个变量的定义，或者这个变量的定义在结构体中，那么我们就返回 Error Type 1。

(3) 错误类型 5, 6：赋值号两边类型不匹配，赋值号左边出现只有右值的表达式。

针对这两种错误，我们需要检索的语法单元是“=”，也就是 ASSIGNOP。我们需要分别检查 ASSIGNOP 的父节点（赋值号左边的内容）以及 ASSIGNOP 的左子节点（赋值号右边的内容），原因同上，我们采用的是二叉树表示多叉树。

针对错误类型 5：我们需要判断左侧是否类型是 INT 或者 FLOAT，如果是，那么这个赋值语句就是形如“1=ax+b”的，那么就是满足错误类型 5 的。

针对错误类型 6：我们需要判断两侧 id 的 basic 是否相同，如果不相同就证明两侧的类型是不匹配的，就要返回错误类型 6。

(4) 错误类型 12：数组访问操作符“[··]”中出现非整数。

每当我们遇到数组访问语句的时候，我们需要对于语法单元进行拆分，以 a[1]为例：我们会将其拆分为 4 个语法单元：“a”，“[”，“1”，“]”。那么显然这种错误对应就是第三个语法单元对应的类型不是 INT 而是 FLOAT。

```
if(right_type->type==BASIC&&right_type->basic==T_INT){
    return left_type->array.elem;
}
else{
    // [float]
    printf("Error type 12 at Line %d: Not integer.\n",child->line);
    return NULL;
}
```

如上图，if 中对应的是第三个语法单元是整数的情况，我们将其视为数组进行后续处理即可。反之 else 中的情况证明第三个语法单元是一个 FLOAT 类型，这样我们就要返回错误类型 12。

综合上述的分析，我们针对每一种错误都是针对其错误情况对应的语法进行拆分，然后针对有可能出现错误的语法单元进行判断即可。

四、 实验的操作流程

我编写了 MakeFile 文件用来进行实验的运行，其中设置了对应语言使用的编译环境以及一些伪目标。文件内容如下：

```
CFILES = $(shell find ./ -name "*.c")
OBJS = $(CFILES:.c=.o)
LFILE = $(shell find ./ -name "*.l")
VFILE = $(shell find ./ -name "*.y")
LFC = $(shell find ./ -name "*.l" | sed s/[/]\*/\./lex.yy.c/)
VFC = $(shell find ./ -name "*.y" | sed s/[/]\*/\./syntax.tab.c/)
LFO = $(LFC:.c=.o)
VFO = $(VFC:.c=.o)

parser: syntax $(filter-out $(LFO),$(OBJS))
$(CC) -o parser $(filter-out $(LFO),$(OBJS)) -lfl

syntax: lexical syntax-c
$(CC) -c $(VFC) -o $(VFO)

lexical: $(LFILE)
$(FLX) -o $(LFC) $(LFILE)

syntax-c: $(VFILE)
$(BISON) -o $(VFC) -d -v $(VFILE)

-include $(patsubst %.o, %.d, $(OBJS))

# 定义的一些伪目标
.PHONY: clean test1 test3 test5 test6 test7 test10 test12 test2 test4 test8 test9 test11 test13 test14 test15 test16 test17
test1:
./parser ../test1.cmm
test3:
./parser ../test3.cmm
test5:
./parser ../test5.cmm
test6:
./parser ../test6.cmm
test7:
./parser ../test7.cmm
test10:
./parser ../test10.cmm
test12:
./parser ../test12.cmm
```

在运行项目的时候首先使用“make parser”命令构建语义分析器，然后使用命令“./parser test1.cmm”来完成对于 test1.cmm 文件的语义分析。

五、 实验的运行结果（仅展示部分）

```
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test1.cmm
Error type 1 at Line 4: Undefined variable "j".
Error type 5 at Line 4: Type mismatched for assignment.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test4.cmm
Error type 4 at Line 6: Redefined function "func".
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test13.cmm
Error type 13 at Line 9: Illegal use of ".".
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test8.cmm
Error type 8 at Line 4: Type mismatched for return.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test5.cmm
Error type 5 at Line 4: Type mismatched for assignment.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test6.cmm
Error type 6 at Line 4: The left-hand side of an assignment must be a variable.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test7.cmm
Error type 7 at Line 4: Type mismatched for operands.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test12.cmm
Error type 12 at Line 4: Not integer.
Error type 5 at Line 4: Type mismatched for assignment.
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$ ./parser test2.cmm
Error type 2 at Line 4: Undefined function "inc".
mb@ubuntu:~/Desktop/Lab2-Michael_Bryant$
```

六、 实验总结

1. 通过本次实验，我对于语法分析树的构建以及作用有了进一步的认识，从实验 2 结束后再回顾实验 1 也有了更深刻的体会。
2. 通过本次实验，我理解了如何从本质上考虑语义分析的错误类型：将语句拆分成一系列语法单元，逐个语法单元查验是不是出现错误，如果出现错误是哪一种类型。也对于课内学习的知识进行了应用与巩固。
3. 通过本次实验，我完成了简单的 C—语言的语义分析任务。